

THÉORIE DES GRAPHS

Séance 3 : Parcours en Profondeur (DFS) Applications avancées

L2 - Génie Informatique • Semestre 3

Ing. GBAGUIDI AISSE Alexandre
ESGC VERECHAGUINE A.K. • Cotonou, Bénin

Plan de la séance



Rappel : Pile

LIFO, opérations

5 min



Problèmes motivants

Labyrinthe, cycles

10 min



DFS Récursif

Profondeur d'abord

20 min



DFS Itératif

Avec pile explicite

15 min



Timestamps

Classification arêtes

20 min



Applications

Cycles, tri topo

15 min



TPE (1h30)

Implémenter DFS

Code C++

Objectifs de la séance

1

Comprendre le principe du DFS

Explorer un graphe couche par couche
Utiliser une pile (LIFO) pour gérer l'ordre de visite

2

Implémenter l'algorithme DFS

Structures : `visited[]`, `discovery[]`, `finish[]`
Versions récursive et itérative

3

Calculer les timestamps

Temps de découverte `discovery[u]`
Temps de fin `finish[u]`

Objectifs de la séance (suite)

4 Classifier les arêtes en 4 types

Tree edges (arbre DFS) | Back edges (cycles)
Forward edges | Cross edges

5 Détecter les cycles

Graphes orientés : chercher les back edges
Graphes non-orientés : arête vers visité (sauf parent)

6 Appliquer DFS à des problèmes réels

Tri topologique : ordonner des tâches
Détection de dépendances circulaires
Composantes fortement connexes

PARTIE 2

Rappel :
Structure de Pile
"Last In, First Out"

Structure de Pile (Stack)

LIFO = Last In, First Out

Opérations principales :

- `push(x)` : Empiler `x` au sommet
- `pop()` : Dépiler et retourner le sommet
- `top()` : Consulter le sommet (sans retirer)
- `is_empty()` : Tester si vide

Exemple :

Pile vide : `[]`

`push(5)` → `[5]`

`push(10)` → `[5, 10]`

`push(3)` → `[5, 10, 3]`

`pop()` → `3`, pile = `[5, 10]`

✓ Complexité : `push()` = $O(1)$, `pop()` = $O(1)$, `top()` = $O(1)$, `empty()` = $O(1)$ — Toutes les opérations en temps constant !

Pile en C++ - std::stack

```
#include <stack>
#include <iostream>
using namespace std;

int main() {
    stack<int> S;

    S.push(5);    // Empiler
    S.push(10);
    S.push(3);

    // Sommet
    cout << S.top() << endl; // 3

    // Dépiler
    while (!S.empty()) {
        cout << S.top() << " ";
        S.pop();
    }
    // Affiche : 3 10 5
}
```



La pile est idéale pour DFS

Toutes les opérations en $O(1)$.

Permet de revenir en arrière facilement après avoir exploré en profondeur.

Lien entre récursivité et pile d'appels

Chaque appel récursif est empilé dans la pile d'appels. Quand une fonction termine, elle est dépilée. C'est le comportement d'une pile LIFO !

RÉCURSIF (pile implicite)

```
void dfs(int u) {  
    visited[u] = true;  
  
    for (int v : adj[u]) {  
        if (!visited[v]) {  
            dfs(v); // Récursif  
        }  
    }  
}
```

ITÉRATIF (pile explicite)

```
void dfs_iter(int src) {  
    stack<int> S;  
    S.push(src);  
  
    while (!S.empty()) {  
        int u = S.top();  
        S.pop();  
        if (!visited[u]) {  
            visited[u] = true;  
            for (int v : adj[u])  
                S.push(v);  
        }  
    }  
}
```

💡 Les deux versions font la même chose ! Récursif utilise la pile d'appels, itératif utilise une pile explicite.

PARTIE 3

Problèmes motivants

"Pourquoi approfondir DFS ?"

Problème 1 : Explorer un labyrinthe



Situation : Vous êtes dans un labyrinthe et voulez trouver la sortie. Quelle stratégie adopter ?



Stratégie DFS

1. Avancer le plus loin possible
2. Si impasse, revenir en arrière
3. Prendre une autre direction
4. Répéter jusqu'à trouver la sortie



Analogie avec graphe

- Intersections = Sommets
- Couloirs = Arêtes
- Avancer = Visiter un sommet
- Revenir = Dépiler de la pile

→ DFS explore en profondeur !



DFS ne garantit PAS le plus court chemin, mais trouve UN chemin si il existe !

Problème 2 : Trouver un chemin entre deux points

? Question : Existe-t-il un chemin entre le sommet A et le sommet B ?



BFS (Séance 2)

Trouve le PLUS COURT chemin

Garantit le minimum d'arêtes

Idéal si vous voulez optimiser



DFS (Séance 3)

Trouve UN chemin (pas forcément le plus court)

Plus simple à implémenter

Suffit si on veut juste savoir si un chemin existe



Exemple Cotonou : Réseau routier

Existe-t-il un itinéraire de Étoile Rouge à Ganhi ? → DFS peut répondre OUI/NON rapidement, sans chercher le plus court.

Problème 3 : Détecter si un graphe contient un cycle



Cycle = Chemin qui revient au point de départ



SANS CYCLE (DAG)

$A \rightarrow B \rightarrow C \rightarrow D$

Pas de retour en arrière possible

Graphe Acyclique Dirigé
(Directed Acyclic Graph)



AVEC CYCLE

$A \rightarrow B \rightarrow C \rightarrow A$

On peut revenir à A !

Problème : dépendance circulaire

💡 **DFS détecte les cycles naturellement : si on visite un sommet DÉJÀ dans la pile d'exploration → CYCLE !**



Applications réelles

- Dépendances circulaires (A attend B, B attend C, C attend A → blocage !)
- Dette circulaire entre commerçants
- Compilation de code (module A dépend de B qui dépend de A)

PARTIE 4

Algorithme DFS Récursif

"Profondeur d'abord"

Principe du DFS : Aller au plus profond



Principe

DFS explore le graphe en allant le plus PROFOND possible avant de revenir en arrière. Comme descendre jusqu'au fond d'une branche d'arbre avant de remonter.



Étapes DFS

1. Marquer le sommet actuel comme visité
2. Pour chaque voisin NON visité :
 - Appeler DFS récursivement
3. Quand tous les voisins sont visités :
 - Revenir au sommet précédent



Métaphore

Imaginez explorer une grotte :

- Vous avancez dans un tunnel jusqu'au bout
- Si impasse, vous revenez à la dernière intersection
- Vous prenez un autre tunnel
- Et ainsi de suite...

C'est exactement DFS !



Différence BFS : BFS explore niveau par niveau (largeur), DFS explore branche par branche (profondeur)

Algorithme DFS_Visit (récursif)



Fonction récursive qui explore un sommet et tous ses descendants

```
DFS_Visit(Graph G, int u, visited[]):  
  
    // 1. Marquer le sommet comme visité  
    visited[u] = true  
    print(u) // Traiter le sommet  
  
    // 2. Explorer tous les voisins  
    for each neighbor v of u:  
        if not visited[v]:  
            DFS_Visit(G, v, visited) // Récursif !  
  
    // 3. Automatique : retour après exploration
```



Idée clé

- Visiter le sommet u
- Pour chaque voisin v :
 - Si non visité,
appeler DFS_Visit(v)
- La récursion gère
automatiquement
le retour en arrière

Algorithme DFS complet



Fonction principale qui lance DFS_Visit depuis tous les sommets non visités

DFS (Graph G) :

```
// 1. Initialiser tous les sommets
for each vertex u in V:
    visited[u] = false

// 2. Lancer DFS_Visit depuis chaque non-visit  
for each vertex u in V:
    if not visited[u]:
        DFS_Visit(G, u, visited)
```

? Pourquoi boucler ?

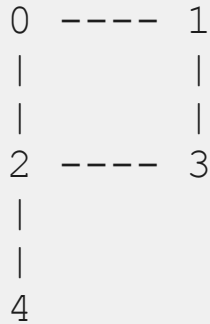
Pour g  rer les graphes NON CONNEXES (plusieurs composantes).

⚡ Complexit  

Temps : $O(V + E)$
Espace : $O(V)$

Exemple DFS : Graphe de départ

Graphe non-orienté :



Informations

- Sommets : $\{0, 1, 2, 3, 4\}$
- Arêtes : $(0-1), (0-2), (1-3), (2-3), (2-4)$
- Source : 0
- Graphe connexe

? Question

Quel sera l'ordre de visite avec DFS depuis le sommet 0 ?

Rappel : On explore en profondeur !

Exemple DFS : Exécution pas à pas



État initial : `visited[] = {false, false, false, false, false}`, `source = 0`

1

`DFS_Visit(0)`

`visited[0] = true`, visiter voisins {1, 2}

2

`DFS_Visit(1)`

`visited[1] = true`, visiter voisins {0✓, 3}

3

`DFS_Visit(3)`

`visited[3] = true`, visiter voisins {1✓, 2}

4

`DFS_Visit(2)`

`visited[2] = true`, visiter voisins {0✓, 3✓, 4}

5

`DFS_Visit(4)`

`visited[4] = true`, voisins {2✓} → FIN



Ordre de visite DFS : 0 → 1 → 3 → 2 → 4

Exemple DFS : Arbre résultant

✓ Ordre de visite : 0 → 1 → 3 → 2 → 4

Arbre DFS :

```
0
|
1
|
3
|
2
|
4
```



Observations

- DFS explore en profondeur
- Arbre linéaire (une branche)
- Ordre différent de BFS
- BFS aurait donné : 0-1-2-3-4
- DFS : 0-1-3-2-4

→ DFS va au bout d'une branche avant de revenir !

PARTIE 5

DFS Itératif

"Avec pile explicite"

Pourquoi une version itérative ?

⚠ Limites de la récursion

- Stack overflow si graphe très profond
- Pile d'appels limitée (~10000 niveaux)
- Moins de contrôle sur l'exécution

✅ Avantages itératif

- Pas de limite de profondeur
- Contrôle explicite de la pile
- Parfois plus efficace en mémoire

💡 **Solution : Remplacer la pile d'appels (récursion) par une pile explicite (`std::stack`)**

📌 Important

L'ordre de visite peut être légèrement différent selon l'ordre d'empilement des voisins, mais les deux versions explorent les mêmes sommets.

Algorithme DFS itératif (avec pile explicite)

```
DFS_Iterative(Graph G, int source):  
  
    // Créer une pile  
    Stack S  
    S.push(source)  
  
    while not S.is_empty():  
        // Dépiler  
        u = S.pop()  
  
        // Si déjà visité, passer  
        if visited[u]:  
            continue  
  
        // Marquer visité  
        visited[u] = true  
        print(u)  
  
        // Empiler voisins NON visités  
        for each neighbor v of u:  
            if not visited[v]:  
                S.push(v)
```



Différence clé

On empile d'abord, puis on marque visité
APRÈS avoir dépilé.

Évite les doublons dans la pile.

↔ Récursif vs Itératif

Récursif : Pile implicite

Itératif : Pile explicite

Même complexité $O(V+E)$

Différence subtile : Ordre de visite

⚠ L'ordre de visite peut différer selon l'ordre dans lequel on empile les voisins. Cela dépend de l'implémentation de la liste d'adjacence.

Récurusif

0 → 1 → 3 → 2 → 4

Suit l'ordre naturel
de la récursion

Itératif

0 → 2 → 4 → 3 → 1

Dépend de l'ordre
d'empilement

💡 **Ce qui compte**
Les deux versions visitent TOUS les sommets et détectent les mêmes propriétés (cycles, connexité, etc.). L'ordre exact n'est pas important pour la plupart des applications.

Quand utiliser récursif vs itératif ?

	Récursif	Itératif
Simplicité du code	✓ Plus simple	Plus verbeux
Graphes très profonds	✗ Stack overflow	✓ Pas de limite
Contrôle fin	Limité	✓ Total
Complexité	$O(V + E)$	$O(V + E)$

🎯 **Recommandation : Utilisez récursif par défaut (plus simple). Passez à itératif si vous avez des graphes très profonds (>10000 niveaux).**

PARTIE 6

Timestamps
et classification
des arêtes

"Horodatage du parcours"

Timestamps : Découverte et Fin

Timestamps : Horodatage du parcours

DFS peut enregistrer deux temps pour chaque sommet : quand on le découvre, et quand on a fini de l'explorer.

discovery[u]

Temps de découverte

Quand on visite u pour la PREMIÈRE fois

Incrémenter un compteur global

finish[u]

Temps de fin

Quand on a fini d'explorer TOUS les voisins de u

Juste avant de revenir en arrière

 **Propriété importante : $\text{discovery}[u] < \text{finish}[u]$ toujours ! (On découvre avant de finir)**

Comment calculer discovery[] et finish[] ?

```
int time = 0 // Compteur global

DFS_Visit_Timestamps(Graph G, int u):

    // Temps de découverte
    time = time + 1
    discovery[u] = time
    visited[u] = true

    // Explorer voisins
    for each neighbor v of u:
        if not visited[v]:
            DFS_Visit_Timestamps(G, v)

    // Temps de fin
    time = time + 1
    finish[u] = time
```



Explications

1. Compteur global time
2. Quand on visite u :
 - time++
 - discovery[u] = time
3. Explorer voisins
4. Après exploration :
 - time++
 - finish[u] = time

Exemple avec timestamps

Graphe :

```
0 → 1
↓   ↓
2 → 3
```

Timestamps après DFS depuis 0 :

Sommet 0 : [1, 8] (découvert en 1, fini en 8)

Sommet 1 : [2, 7] (découvert en 2, fini en 7)

Sommet 3 : [3, 6] (découvert en 3, fini en 6)

Sommet 2 : [4, 5] (découvert en 4, fini en 5)

Ordre de visite : $0 \rightarrow 1 \rightarrow 3 \rightarrow 2$

✓ Observation

Les intervalles [discovery, finish] s'emboîtent comme des parenthèses ! Si u est ancêtre de v , alors $[discovery[u], finish[u]]$ contient $[discovery[v], finish[v]]$.

Classification des arêtes (graphes orientés)



DFS permet de classer chaque arête en 4 types selon les timestamps

Tree Edge

Arête de l'arbre DFS
Vers sommet NON visité

→ *discovery[v]* est défini après *u*

Back Edge

Vers un ANCÊTRE
Indique un CYCLE !

→ *v* est ancêtre de *u*

Forward Edge

Vers un DESCENDANT
(non direct)

→ *v* est descendant de *u*

Cross Edge

Vers un autre
sous-arbre

→ Ni ancêtre ni descendant

PARTIE 7

Applications du DFS

"Cycles, tri topologique, SCC"

Application 1 : Détection de cycle

Principe

Un cycle existe si et seulement si DFS rencontre une BACK EDGE (arête vers un ancêtre dans l'arbre DFS)

Graphe orienté

Détection simple :

- Pendant DFS, maintenir un ensemble de sommets "en cours"
- Si on visite un sommet "en cours" → BACK EDGE → CYCLE !
- Sinon → PAS de cycle (DAG)

Graphe non-orienté

Détection :

- Si on visite un voisin déjà visité (et ce n'est PAS le parent direct) → CYCLE
- Chaque arête existe dans les 2 sens, donc ignorer le parent

 **Complexité : $O(V + E)$ — DFS détecte les cycles en un seul parcours !**

Algorithme : Détection de cycle (graphe orienté)

```
bool has_cycle = false
visited[V] = {false}
in_stack[V] = {false} // Sommets en cours

DFS_Cycle(Graph G, int u):

    visited[u] = true
    in_stack[u] = true // Ajouter à la pile

    for each neighbor v of u:
        if not visited[v]:
            DFS_Cycle(G, v)
        else if in_stack[v]:
            // Back edge détectée !
            has_cycle = true

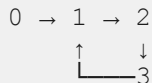
    in_stack[u] = false // Retirer de la pile
```

Points clés

- in_stack[] : sommets dans le chemin actuel
- Si voisin v est in_stack → on revient à un ancêtre → CYCLE
- Retirer de in_stack[] après exploration complète

Exemple : Graphe avec cycle

Graphe orienté :



Arêtes :

0→1, 1→2, 2→3, 3→1

CYCLE : 1 → 2 → 3 → 1



Exécution DFS depuis 0

1. Visiter 0, in_stack = {0}
2. Visiter 1, in_stack = {0, 1}
3. Visiter 2, in_stack = {0, 1, 2}
4. Visiter 3, in_stack = {0, 1, 2, 3}
5. Voisin de 3 = 1
 - 1 est in_stack ✓
 - BACK EDGE !
 - CYCLE DÉTECTÉ !



Résultat

DFS détecte le cycle 1 → 2 → 3 → 1. L'arête 3→1 est une BACK EDGE car 1 est un ancêtre de 3 dans l'arbre DFS.

Exemple : Graphe sans cycle (DAG)

DAG (Directed Acyclic Graph) :

```
0 → 1 → 3
↓   ↓
2 → 4
```

Arêtes :

0→1, 0→2, 1→3, 1→4, 2→4

PAS de cycle ✓



Exécution DFS depuis 0

1. Visiter 0
2. Visiter 1
3. Visiter 3 (fini)
4. Visiter 4 (fini)
5. Retour à 0
6. Visiter 2
7. Voisin de 2 = 4 (déjà visité)
 - 4 n'est PAS in_stack
 - Pas de back edge



Aucun cycle détecté



DAG = Directed Acyclic Graph

Un DAG est un graphe orienté SANS cycle. C'est une structure très importante car elle permet le TRI TOPOLOGIQUE (voir slide suivante) !

Application 2 : Tri topologique



Définition : Tri topologique

Ordonner les sommets d'un DAG tel que pour toute arête $u \rightarrow v$, u apparaît AVANT v dans l'ordre.



Cas d'usage

- Prérequis de cours
(Algo 1 avant Graphes)
- Ordre de compilation
(modules avec dépendances)
- Planification de tâches
(construction maison)
- Recettes de cuisine
(étapes ordonnées)



Algorithme simple

1. Faire un DFS complet
2. Trier les sommets par `finish[]` DÉCROISSANT
3. C'est tout !



Condition : Le tri topologique existe SI ET SEULEMENT SI le graphe est un DAG (pas de cycle) !

Algorithme : Tri topologique

```
List topo_order // Liste vide

DFS_Topo(Graph G, int u):

    visited[u] = true

    for each neighbor v of u:
        if not visited[v]:
            DFS_Topo(G, v)

    // Après avoir exploré tous les voisins
    topo_order.push_front(u) // Ajouter au DÉBUT

Topological_Sort(Graph G):
    for each vertex u in V:
        if not visited[u]:
            DFS_Topo(G, u)

    return topo_order
```



Idée clé

Quand on FINIT un sommet, on l'ajoute au DÉBUT de la liste.

Le dernier fini sera le premier dans l'ordre topologique !



Propriété

Pour $u \rightarrow v$:
 $\text{finish}[u] > \text{finish}[v]$

Donc u avant v dans l'ordre topologique ✓

Exemple : Prérequis de cours



Contexte : Programme L2 Génie Informatique — Dans quel ordre suivre les cours ?

Graphe des prérequis :

Algo1 → Algo2 → Graphes
↓ ↓
BD → Web

Dépendances :

- Algo2 nécessite Algo1
- Graphes nécessite Algo2
- BD nécessite Algo1
- Web nécessite Algo2 et BD



Tri topologique

Ordre possible :

1. Algo1
2. Algo2
3. BD
4. Web
5. Graphes

✓ Tous les prérequis respectés !

Applications réelles du tri topologique



Construction

Ordre des tâches pour construire une maison :

- Fondations avant murs
- Murs avant toit
- Électricité avant peinture



Compilation

Ordre de compilation des modules :

- Module A dépend de B
- Compiler B d'abord
- Détecter dépendances circulaires



Recette

Étapes d'une recette :

- Préparer ingrédients
- Cuire dans l'ordre
- Assemblage final



Gestion projet

Planification de tâches :

- Certaines tâches bloquent d'autres
- Trouver l'ordre optimal
- Optimiser le temps



Partout où il y a des dépendances, le tri topologique est utile !

Application 3 : Composantes fortement connexes



Composante Fortement Connexe (SCC)

Sous-ensemble de sommets où on peut aller de n'importe quel sommet à n'importe quel autre (chemin dans les DEUX sens)

Exemple :

$[0 \rightleftharpoons 1] \rightarrow [2 \rightleftharpoons 3]$

SCC 1 : {0, 1}

SCC 2 : {2, 3}

On peut aller de 0→1 et 1→0

On peut aller de 2→3 et 3→2

Mais pas de 1→2 ET 2→1



Algorithme de Kosaraju

1. DFS sur G, calculer finish[]
2. Créer graphe transposé G^T
(inverser toutes les arêtes)
3. DFS sur G^T par ordre décroissant de finish[]
4. Chaque arbre DFS = 1 SCC



Algorithme avancé ! Détails en L3. Complexité : $O(V + E)$ — utilise DFS deux fois.

PARTIE 8

Analyse de
complexité
"Temps et espace"

Analyse de complexité DFS

Complexité temporelle

$$O(V + E)$$

- Chaque sommet visité 1 fois $\rightarrow O(V)$
- Chaque arête explorée 1 fois $\rightarrow O(E)$
- Total : $O(V + E)$

Complexité spatiale

$$O(V)$$

- visited[] : $O(V)$
- Pile d'appels (récursif) : $O(V)$
- Pile explicite (itératif) : $O(V)$
- Total : $O(V)$



Complexité des applications

- Détection de cycle : $O(V + E)$
- Tri topologique : $O(V + E)$
- Composantes connexes : $O(V + E)$
- Composantes fortement connexes (Kosaraju) : $O(V + E)$

→ Toutes les applications DFS sont en $O(V + E)$!

Comparaison BFS vs DFS

Critère	BFS (Séance 2)	DFS (Séance 3)
Structure	File FIFO	Pile LIFO / Récursion
Exploration	Niveau par niveau	Profondeur d'abord
Plus court chemin	✓ Oui (non pondéré)	✗ Non
Détection cycle	Possible	✓ Plus naturel
Tri topologique	✗ Non	✓ Oui (avec finish[])
Applications	Distance, connexité, biparti	Cycle, tri topo, SCC
Complexité temps	$O(V + E)$	$O(V + E)$
Complexité espace	$O(V)$	$O(V)$
Quand utiliser ?	Plus court chemin	Cycle, ordre, dépendances

PARTIE 9

Récapitulatif

"Ce qu'il faut retenir"

Récapitulatif de la séance

1

DFS = Profondeur d'abord

Explore le plus loin possible avant de revenir
Utilise une pile (LIFO) ou récursion

2

Deux versions : récursif / itératif

Récursif : plus simple, pile implicite
Itératif : pas de limite, pile explicite

3

Timestamps discovery[] et finish[]

Permettent de classifier les arêtes
Tree, Back, Forward, Cross edges

4

Applications puissantes

Détection de cycles (back edges)
Tri topologique (order finish[])
Composantes fortement connexes

Quand utiliser DFS ?

Chercher le plus court chemin ?

 Non → Utiliser BFS

Détecter un cycle ?

 Oui → DFS (back edges)

Ordonner avec dépendances ?

 Oui → DFS (tri topologique)

Trouver composantes connexes ?

 Oui → BFS ou DFS (équivalent)

Graphe très profond (>10000) ?

 DFS itératif (pas récursif)

Explorer toutes les branches ?

 Oui → DFS parfait

Tableau récapitulatif : Applications DFS

Application	Principe	Complexité	Contexte
Détection cycle (orienté)	Chercher back edges avec in_stack[]	$O(V + E)$	Dépendances circulaires
Détection cycle (non-orienté)	Visiter voisin déjà visité ($\neq$ parent)	$O(V + E)$	Réseaux physiques
Tri topologique	Ordre décroissant de finish[]	$O(V + E)$	Prérequis, compilation
Composantes connexes	Compter nombre de DFS lancés	$O(V + E)$	Groupes isolés
SCC (Kosaraju)	2 DFS + graphe transposé	$O(V + E)$	Réseaux sociaux